



INTENSIVÃO GAMEDEV

Aula 2 - Personagens, Scripts e Sinais

Gustavo Moraes - @moraguma



GAMUX

FAZENDO JOGO



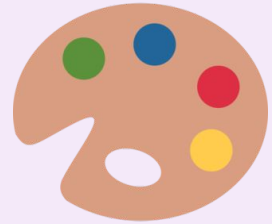
Ideia



Estrutura



Função



Estética

Aula 1

Aula 2

Aula 3

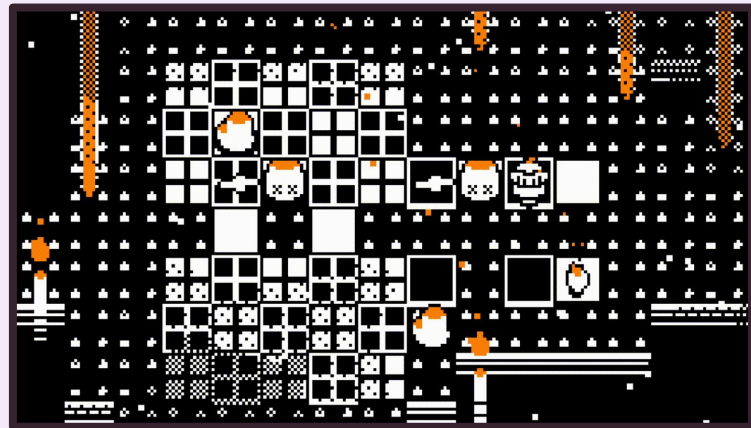


GAMUX

INTRODUÇÃO

Como dar comportamento para cenas por meio de código?

- **Vamos começar com bases de programação**
- **Depois, como aplicar esses conceitos em um jogo**



PROGRAMAÇÃO

```
func _init():  
    print("Hello, world!")
```

Código é uma sequência de passos

- **GScript é a linguagem padrão do Godot**

```
var velocity = Vector2.UP.rotated(rotation) * speed  
position += velocity * delta
```



VARIÁVEIS

Uma variável guarda valores

- Números, palavras ou nós
- É declarada e depois pode ser usada
- Para declarar uma variável, usamos **var**

```
# Declara a variável vidas
# e dá a ela o valor 3
var vidas = 3

# Declara uma nova variável
# e subtrai ela de vidas.
# Agora o valor de vidas
# é 2
var vidas_perdidas = 2
vidas = vidas - vidas_perdidas
```



CONDIÇÕES

```
if vidas == 0:  
>| # Imprime "Você morreu!"  
>| # na tela caso o jogador  
>| # tenha 0 vidas  
>| print("Você morreu!")  
else:  
>| # Caso contrário, imprime  
>| # "Você está vivo!"  
>| print("Você está vivo!")
```

Mudam o fluxo do código com base em checagens

- Declaramos condições com **if** e **else**
- Indentações definem os blocos de código
- Lembre de operadores como **>**, **<**, **>=**, **<=**, **==**, **!=**



FUNÇÕES

Bloco de código que pode ser chamado e executado

- **Faz alguma coisa**
- **Retorna alguma coisa**
- **Pode receber parâmetros**
- **Declarada com `func`**
- **Retorna com `return`**

```
# Dada a quantidade atual de vidas do jogador e
# a quantidade de vidas que ele deve perder,
# imprime se ele ainda está vivo e retorna a sua
# nova quantidade de vidas
func perder_vidas(vidas, vidas_a_perder):
>|  var vidas_final = vidas - vidas_a_perder
>|
>|  if vidas_final <= 0:
>|    >|  print("Você morreu!")
>|  else:
>|    >|  print("Você ainda está vivo!")
>|
>|  return vidas_final

func _ready():
>|  var vidas = 3
>|  var vidas_a_perder = 2
>|
>|  # Vidas guarda o valor de vidas_final que a
>|  # função retornou. Agora o valor de vidas é
>|  # 3 - 2 = 1
>|  vidas = perder_vidas(vidas, vidas_a_perder)
```



**Uma variável
declarada em uma
função só existe
dentro da função**

**É possível declarar
variáveis globais fora
de uma função**

```
var vidas = 3
```

```
# Dada a quantidade de vidas que o jogador deve  
# perder, modifica a variável global vidas e  
# verifica se o jogador está morto
```

```
func perder_vidas(vidas_a_perder):
```

```
>|   vidas = vidas - vidas_a_perder
```

```
>|
```

```
>|   if vidas <= 0:
```

```
>|     >|   print("Você morreu!")
```

```
>|   else:
```

```
>|     >|   print("Você ainda está vivo!")
```

```
func _ready():
```

```
>|   var vidas_a_perder = 2
```

```
>|
```

```
>|   # A função modifica diretamente a variável
```

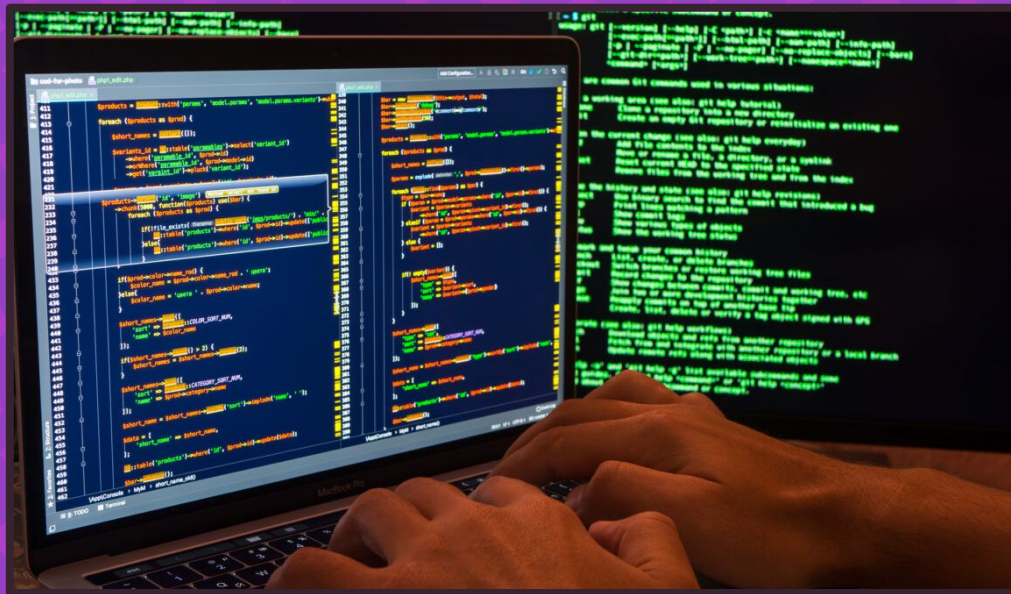
```
>|   # global vidas. Agora vidas = 3 - 2 = 1
```

```
>|   perder_vidas(vidas_a_perder)
```



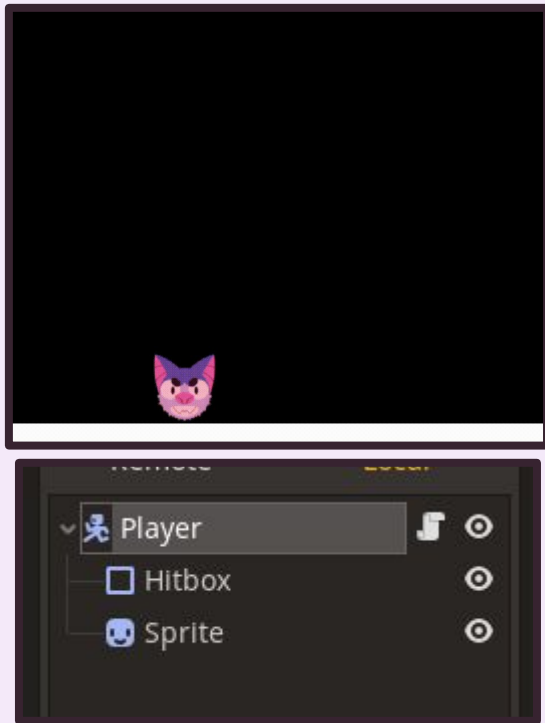
Calma!

- **Vamos aprender a ideia**
- **Como escrever código é secundário**



GAMUX

SCRIPTS



Código que pode ser ligado a nós para gerar funcionalidade

- **Todo nó tem funções e variáveis próprias**
- **Ao criar um script, criamos novas funcionalidades em cima das que o nó já tem**



CharacterBody2D tem
uma função
move_and_slide

- Definimos velocidade
- Chamamos
move_and_slide
- O personagem se

move

```
velocity = Vector2(100, 0)  
move_and_slide()
```



SCRIPTS

Lembre da hierarquia!

- Um nó pode herdar funções e atributos de outro
- *Sprite2D* é um *Node2D*, então ele tem posição (x, y) e rotação
- Todo nó é um *Node*, então tem funções que todo nó tem



`_ready()`

- Chamada quando o nó entra em cena
- Inicializa o nó

`_process(delta)`

- Chamada a cada frame
- *delta* é o tempo entre frames

`_physics_process(delta)`

- Chamada a cada frame de física
- Igual ao process, mas com FPS fixo



PLANEJANDO

**Botões
Ambiente**



**Se mexer
Coletar item
Morrer**

Pense no que quer fazer antes de programar!

- **Código gera comportamento com base na entrada**
- **O que, exatamente, seu jogo deve fazer?**

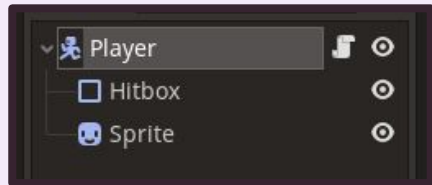
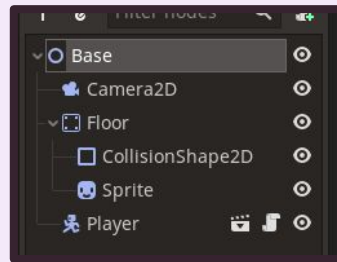
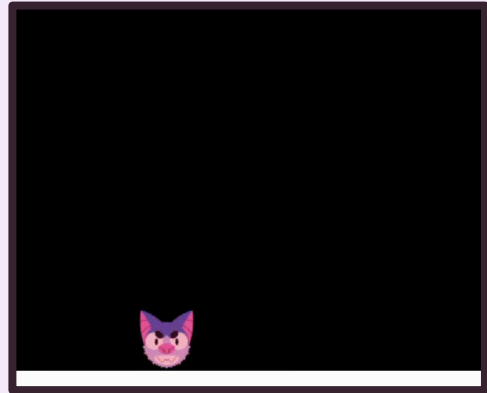


GAMUX

PROCESSOS CONTÍNUOS

O Gamuto...

- **Responde aos botões apertados**
 - **Esquerda e direita andam**
 - **Espaço pula**
- **É afetado pela gravidade**
- **Vamos lembrar de física!**



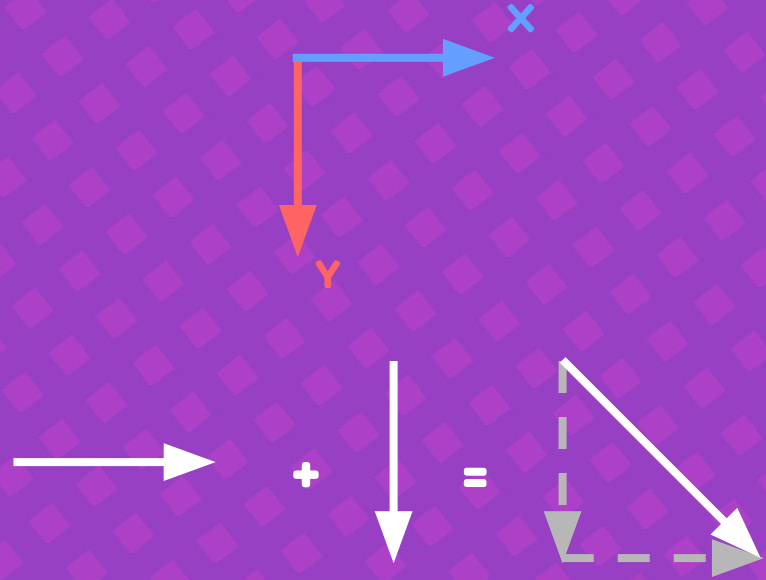
Vetores nos permitem pensar em movimento

As componentes da velocidade:

- Movimento (eixo x)
- Gravidade (eixo y)

Velocidade é a soma de v_x e v_y

A cada momento, calculamos a velocidade e movemos o jogador



Passo a passo

- **Antes de pensar no código, entendemos o que deve acontecer**

Se “direita” apertado:

Dar velocidade em $+x$

Se “esquerda” apertado:

Dar velocidade em $-x$

Se nenhum apertado:

Zerar velocidade no eixo x

Se “ação” apertado e estiver no chão:

Dar velocidade em $-y$

Aplicar a gravidade na velocidade

Mover de acordo com a velocidade



Vamos escrever o código!

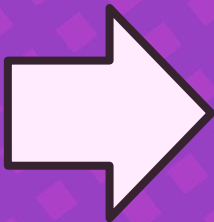
- **Utilizaremos *_physics_process* para calcular a velocidade a cada frame**

Isso é um exemplo! Não precisa decorar



***velocity* vem de
CharacterBody2D e define a
velocidade**

**Se “direita” apertado:
Dar velocidade em +x
Se “esquerda” apertado:
Dar velocidade em -x
Se nenhum apertado:
Zerar velocidade no eixo x**

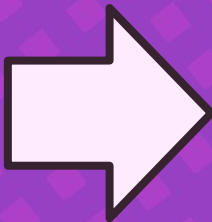


```
func _physics_process(delta):  
    >| if Input.is_action_pressed("direita"):  
    >|     velocity.x = 300  
    >| elif Input.is_action_pressed("esquerda"):  
    >|     velocity.x = -300  
    >| else:  
    >|     velocity.x = 0
```



***is_on_floor* vem de
CharacterBody2D e retorna se
está no chão**

**Se “ação” apertado e estiver no chão:
Dar velocidade em -y**



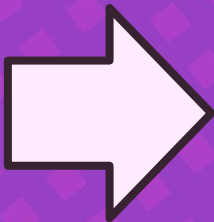
```
if Input.is_action_just_pressed("acao") and is_on_floor():  
    velocity.y = -600
```



As fórmulas da física ajudam a calcular movimento!

$$\Delta \mathbf{v} = \mathbf{a} . \Delta t$$

Aplicar a gravidade

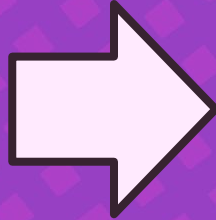


```
velocity.y += 800 * delta
```



move_and_slide* do *CharacterBody2D
move de acordo com *velocity*

Mover de acordo com a velocidade



```
move_and_slide()
```



Se “direita” apertado:

Dar velocidade em +x

Se “esquerda” apertado:

Dar velocidade em -x

Se nenhum apertado:

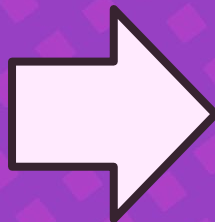
Zerar velocidade no eixo x

Se “ação” apertado e estiver no chão:

Dar velocidade em -y

Aplicar a gravidade na velocidade

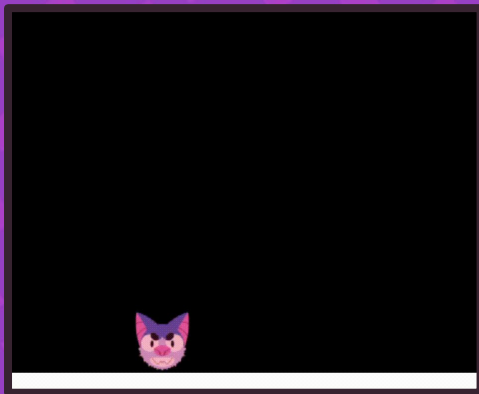
Mover de acordo com a velocidade



```
func _physics_process(delta):  
    if Input.is_action_pressed("direita"):  
        velocity.x = 300  
    elif Input.is_action_pressed("esquerda"):  
        velocity.x = -300  
    else:  
        velocity.x = 0  
  
    if Input.is_action_just_pressed("acao") and is_on_floor():  
        velocity.y = -600  
  
    velocity.y += 800 * delta  
  
    move_and_slide()
```



```
# func _physics_process(delta):  
#     if Input.is_action_pressed("direita"):  
#         velocity.x = 300  
#     elif Input.is_action_pressed("esquerda"):  
#         velocity.x = -300  
#     else:  
#         velocity.x = 0  
#  
#     if Input.is_action_just_pressed("acao") and is_on_floor():  
#         velocity.y = -600  
#  
#     velocity.y += 800 * delta  
#  
#     move_and_slide()
```



Esse jogo não é perfeito

- Vários valores repetidos
- Gamuto não acelera
- Não considera controles analógicos

A forma da sua implementação depende da sua visão pro jogo!

EVENTOS

Algumas coisas devem acontecer em resposta a eventos

- **Quando tocar em moeda**
 - **+1 ponto**
 - **Moeda desaparece**
- **No Godot, fazemos isso com uma combinação de técnicas**



CHAMADAS DE OBJETO



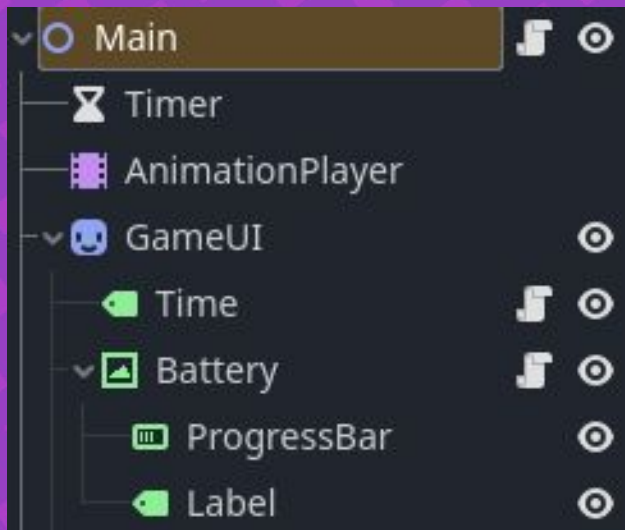
```
var timer

func _ready():
    > timer = get_node("Timer")
    > timer.start(5)
```

Chamando funções em outros nós

- Guardamos nós em variáveis
 - *get_node(path)* pega um nó a partir de um caminho
- Podemos acessar funções e variáveis desse nó com um **.**
- No exemplo, iniciamos o nó *Timer* a partir do nó *Deerskull*





Exercícios!

Como iniciar *Timer* a partir de *Main*?

```
var timer = get_node("Timer")  
timer.start(5)
```

Como obter *Battery* de *Main*?

```
get_node("GameUI/Battery")
```

E *ProgressBar* de *Battery*?

```
get_node("ProgressBar")
```

E *ProgressBar* de *Time*?

```
get_node("../Battery/Progress  
Bar")
```



Também podemos usar $\$$ no lugar de `get_node()`!

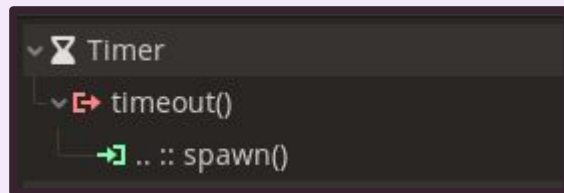
`get_node("Timer")` $\Leftrightarrow$ `$Timer`



SINAIS

Chamam funções quando ocorrem eventos

- *timeout* é emitido quando um *Timer* chega em zero
- Conectamos a emissão de um sinal à chamada de uma função
- Quando *timeout* é emitido, chamamos *spawn*



```
43 var pontuacao = 0
44
45 func spawn():
46   >| pontuacao += 1
47   >|
48   >| timer.start(5)
```



Sinais funcionam em combinação com chamadas de objeto!

Para criar um tiro que causa dano em um inimigo

- **Sinal é emitido quando tiro toca em um inimigo**
- **É conectado a uma função que causa dano no inimigo e deleta o tiro**



Dois aspectos na programação de jogos

Momento-a-momento

- Acontece todo frame
- Movimento, processamento de inputs
- Implementado com `_process` e `_physics_process`

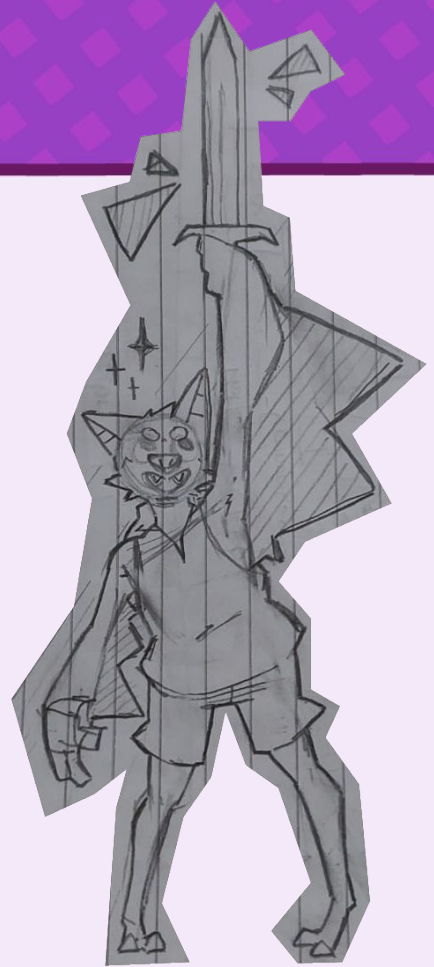
Eventos

- Acontecem em resposta a eventos
- Levar dano ao tocar em inimigo, bomba explodindo após *Timer* terminar
- Implementado com chamadas de objeto e sinais



FAZENDO JOGOS

**Vamos praticar! Hora de pensar
na implementação de alguns
jogos**





Podemos andar para esquerda e direita e, quando no chão, inverter a gravidade. Além disso, tocar em espinhos te mata

Momento-a-momento

**se “esquerda” apertado:
dar velocidade em $-x$
se “direita” apertado:
dar velocidade em $+x$**

**se “gravidade” apertado e estiver no chão:
inverter gravidade**

aplicar gravidade na direção da gravidade

mover de acordo com a velocidade

Eventos

**Quando tocar em um espinho:
Matar personagem**



GAMUX

Momento-a-momento

- se “esquerda” apertado:
dar velocidade em $-x$
- se “direita” apertado:
dar velocidade em $+x$
- se “cima” apertado:
dar velocidade em $-y$
- se “baixo” apertado:
dar velocidade em $+y$

mover de acordo com a velocidade

- se “atirar_esquerda” apertado:
criar tiro com velocidade $-x$
- se “atirar_direita” apertado:
criar tiro com velocidade $+x$
- se “atirar_cima” apertado:
criar tiro com velocidade $-y$
- se “atirar_baixo” apertado:
criar tiro com velocidade $+y$



Podemos andar para as quatro direções e, independentemente, atirar para as quatro direções



GAMUX



**Temos um limite de tempo que,
quando acabar, morremos.
Podemos ganhar mais tempo
matando inimigos**

Eventos

**Quando começar o jogo:
Iniciar o timer com um tempo inicial**

**Quando matar um inimigo:
Dar mais tempo para o timer**

**Quando acabar o timer:
Matar o jogador**



GAMUX

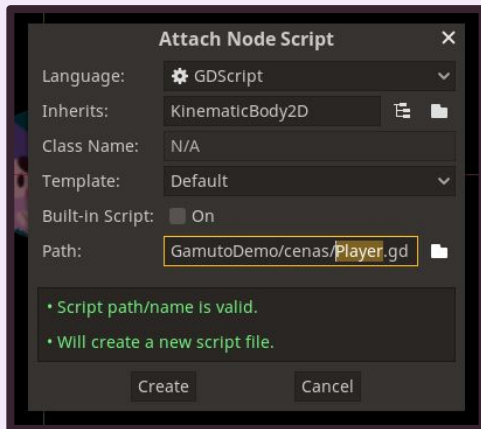
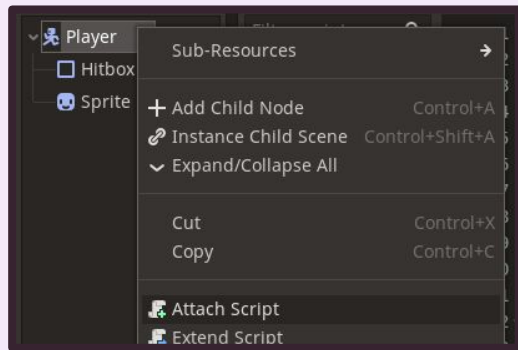
FAZENDO JOGOS

É normal ter dificuldade de pensar no código

- **Não tem como decorar centenas de nós e funções**
- **A ideia é a parte importante**
- **Use a documentação e peça ajuda! É o que todo mundo faz**



NA PRÁTICA



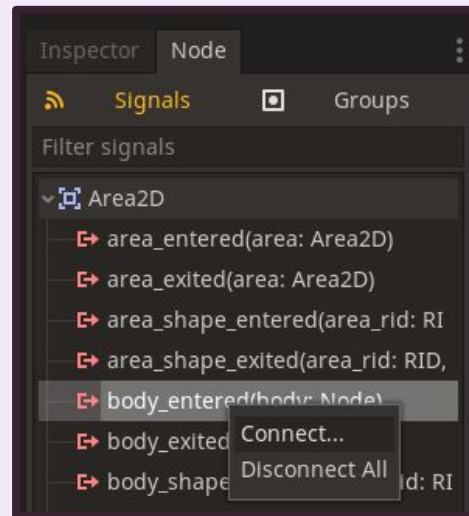
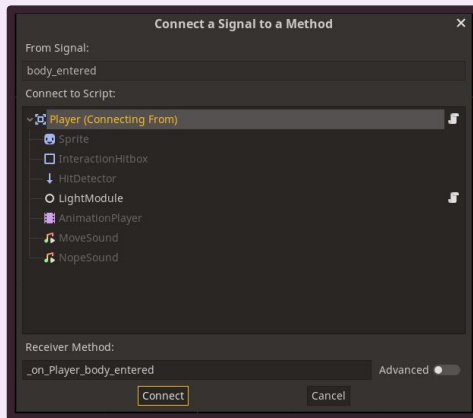
Para adicionar um script a um nó, aperte nele com o botão direito

- **Lembre de modificar a path, para escolher onde seu script será salvo e qual será o nome dele**

NA PRÁTICA

Para conectar um sinal a uma função, entre no inspetor e clique com o botão direito em um sinal

- **Escolha a função para conectar**



NA PRÁTICA

```
3
4  signal win
5  signal lose
6
7
8  const WIDTH = 1920
9  const HEIGHT = 1080
10
```

O script da cena principal

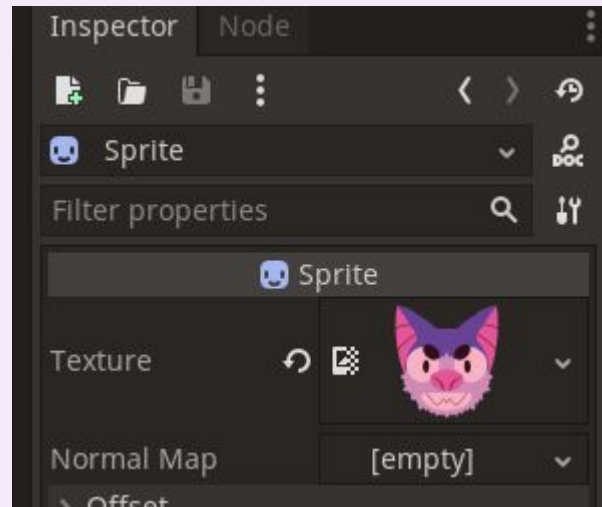
- **WIDTH e HEIGHT** definem a resolução do seu jogo
- ***win* e *lose*** definem se o jogador venceu o jogo. Use ***win.emit()* e *lose.emit()***



NA PRÁTICA

Para dar uma textura para um Sprite, abra seu inspetor e arrastar uma imagem do visualizador de arquivos para a parte de textura

- **Coloque alguma coisinha aqui, mesmo antes de ter as texturas finais**





**Você já tem o necessário para fazer
seu jogo funcionar**

**Tente implementar sua ideia. Amanhã
vamos ver como polir o seu jogo!**



GAMUX